

CosmoRobot

Bitácora técnica de la sesión de construcción — 10 y 11 de julio de 2026

Documento de referencia inicial del proyecto. Registra qué se hizo, cómo se hizo, qué tuvo que corregirse, y qué se aprendió — incluyendo de las partes que salieron bien. Escrito para que lo entienda alguien que no programa, sin perder precisión técnica para quien sí lo hace.

1. Por qué CosmoRobot no es un robot "tradicional"

Para entender por qué vale la pena documentar esta sesión con tanto detalle, hay que entender primero en qué se diferencia CosmoRobot de un robot construido de la manera habitual.

La mayoría de los robots — incluidos casi todos los que se arman con un kit LEGO Mindstorms siguiendo un tutorial — se diseñan según lo que podríamos llamar el **paradigma de Shannon**, en honor a Claude Shannon, el ingeniero que sentó las bases matemáticas de la teoría de la información en los años 40. En ese paradigma, un robot es, en esencia, una tubería:

1. **Sensor:** mide algo del mundo (una distancia, una luz, un contacto).
2. **Programa:** un conjunto de reglas escritas de antemano por el diseñador, del tipo "si la distancia es menor a X, hacé Y".
3. **Actuador:** ejecuta la acción Y.

Este enfoque es extremadamente poderoso y responsable de casi toda la robótica industrial del mundo. Pero tiene una característica central: **el comportamiento del robot está completamente decidido antes de que el robot exista**. El programador ya sabe, para cada situación posible, qué va a hacer la máquina. El robot no "decide" nada en el sentido fuerte de la palabra — ejecuta un plan. Si el robot repite siempre la misma acción ante la misma situación, eso no es un defecto: es el objetivo. Un buen robot Shannon es *previsible* y *óptimo*: hace lo correcto, siempre, de la misma manera, con el mínimo desperdicio de energía o movimiento.

CosmoRobot está construido bajo una teoría distinta — la **Teoría Cosmosemiótica** de Alexis — que parte de una pregunta distinta: no "¿qué debe hacer la máquina en cada situación?", sino "¿cómo se construye un organismo que tenga una relación propia con su situación, en vez de una tabla de respuestas?". Las diferencias concretas, todas visibles en el código que se construyó y corrigió esta sesión, son:

- **No hay una tabla de "si pasa X, hacer Y" para el movimiento normal.** Hay un módulo — el organelo de deliberación — que en cada ciclo *pesa* varias opciones de movimiento según dos cosas: qué tan bien le fue a esa opción en el pasado reciente (una memoria de valencia, algo parecido a "esto me gustó" o "esto me costó caro"), y qué tan urgente es la situación actual. La opción elegida no está escrita en ningún lado de antemano: emerge de ese cálculo, ciclo a ciclo.
- **El robot tiene un estado interno que se parece a "cómo se siente".** El organelo de propiocepción calcula, a partir de señales genéricas del cuerpo (error respecto a lo que busca, energía de batería, presión de tener que escapar de un aprieto), un número de bienestar y otro de malestar. Estos números no mueven motores directamente, pero alimentan la memoria de valencia y, como se agregó esta misma noche, la intensidad de la respuesta de emergencia. El robot no solo mide el mundo: se mide a sí mismo.
- **Las acciones nunca son "óptimas" a propósito.** Cada vez que el robot decide moverse, la potencia y la duración exactas se sortean dentro de un rango, no son un valor fijo calculado para minimizar el esfuerzo. Esto es un principio de diseño explícito (el "Principio de Reserva Estructural"): un organismo que siempre hace lo más eficiente no tiene margen para descubrir usos nuevos de sus propias capacidades. Un robot Shannon optimiza; CosmoRobot deja "sobras" a propósito.
- **Tiene memoria de lo malo, no solo de lo bueno.** Si una acción llevó a un choque, esa opción queda marcada como "trauma" en una memoria episódica y su puntaje se hunde con fuerza durante un tiempo, sin importar cuánto la favoreciera la valencia acumulada. Es una negación que domina cualquier cálculo de conveniencia — parecido a cómo un animal no vuelve a probar algo que le hizo daño, aunque en teoría "convenga".

- **El comportamiento observable no fue diseñado línea por línea: emergió, y a veces sorprendió a sus propios creadores.** El ejemplo más claro de esta sesión: nadie escribió en ningún lado "el robot debe preferir girar a la derecha". Sin embargo, analizando los 20 registros de actividad de la noche, el robot giró a la derecha el 58% de las veces contra un 26% a la izquierda. Ese patrón salió de la combinación entre un desempate al azar al arrancar cada corrida y un mecanismo de "inercia" que refuerza repetir la última decisión — posiblemente reforzado, además, por una asimetría física real entre los dos motores que todavía no se midió. Es un comportamiento que hubo que *descubrir mirando los datos*, no algo que se pueda leer de antemano en el código.

En resumen: un robot Shannon ejecuta un programa. CosmoRobot sostiene una relación con su entorno, mediada por memoria, urgencia y un estado interno propio, dentro de un cuerpo con capacidades que ni siquiera usa siempre de la misma manera. Por eso esta bitácora no es solo una lista de arreglos técnicos: es el registro de las primeras horas de vida de un organismo, no solo de la puesta a punto de una máquina.

2. Arquitectura del robot, en criollo

CosmoRobot corre en dos partes: un **cuerpo** (el ladrillo LEGO Mindstorms NXT, con sus sensores y motores) y una **mente** (un programa en Python que corre en la computadora y le habla al cuerpo por Bluetooth, sin cables). Esta es una decisión de diseño importante tomada en la sesión anterior a esta: en vez de programar el robot con los bloques gráficos oficiales de LEGO (NXT-G), se decidió reutilizar en Python la misma "mente" cosmo-semiótica que ya estaba desarrollada y probada en otro proyecto anterior (ANIMA), evitando reinventarla desde cero.

El código está organizado en "órganos" (carpeta organelos/), cada uno con una responsabilidad chica y clara: un órgano por cada sensor (ultrasónico, EOPD, Color, y el multiplexor que agrupa Touch/Gyro/Acelerómetro/Compass), uno para los motores, uno para la deliberación (la mente), uno para la propiocepción (cómo se siente), y uno nuevo agregado esta noche para la energía de la batería. Un archivo central (main.py) simplemente los conecta en un ciclo que se repite todo el tiempo que el robot esté encendido:

1. **D (Detección):** se leen todos los sensores.
 2. **Evaluación de cambio:** se calcula cuánto cambió el estado del mundo respecto al ciclo anterior, combinando todos los sensores a la vez (no solo uno).
 3. **Capa reactiva:** si hay un obstáculo demasiado cerca adelante o un golpe detectado atrás, el robot frena y se aparta *sin pasar por la deliberación* — es una reacción de protección física, no una decisión meditada.
 4. **R → Acción (deliberación):** si no hubo que reaccionar, la mente elige cuánto y hacia dónde girar.
 5. **Ejecución:** se mueve, con potencia y duración sorteadas dentro de un rango.
 6. **A (Actualización):** se vuelve a medir, se calcula qué tan bien salió, y esa experiencia se guarda en la memoria de valencia.
 7. **Registro:** todo lo anterior — cada sensor, cada cálculo, cada decisión — se escribe en un archivo de registro (datalog) para poder revisarlo después.
-

3. Punto de partida de esta sesión

Al empezar esta sesión, el programa completo ya estaba escrito (de una sesión anterior, el 9-10 de julio): los siete sensores, la capa reactiva, la deliberación con memoria, el motor diferencial, y la conexión por Bluetooth sin cable. Pero, por decisión explícita de Alexis, se había construido todo de una vez y sin probar cada pieza por separado antes de avanzar — así que **la primera corrida física real, completa, con todos los sensores y los motores a la vez, todavía no se había hecho**. Esta sesión fue, entonces, la primera puesta a prueba real del organismo completo contra el mundo físico. Como se detalla más abajo, esa puesta a prueba encontró — y permitió corregir — varios problemas que ninguna revisión de código, por cuidadosa que fuera, habría encontrado sin datos reales.

4. Bitácora cronológica de lo hecho

4.1 — El obstáculo inesperado: hacer hablar a la PC con el robot por Bluetooth

Antes de poder probar nada del robot en sí, hubo que resolver un problema de infraestructura que consumió buena parte de la noche: lograr que el software oficial de LEGO (NXT-G), usado como herramienta de diagnóstico en paralelo al programa Python, pudiera conectarse al robot por Bluetooth.

La causa resultó no tener nada que ver con el robot ni con el código del proyecto, sino con el propio sistema operativo Windows de la computadora: al principio de la sesión se había cambiado el controlador (driver) del puerto USB del robot para que Python pudiera hablarle directamente (un controlador llamado WinUSB, instalado con una herramienta llamada Zadig). Ese cambio, sin embargo, dejaba fuera de juego al software oficial de LEGO, que espera el controlador original del fabricante. Se revirtió ese cambio (recuperando el controlador original "Fantom" desde el propio almacén de controladores de Windows, sin necesidad de reinstalar nada), y — sorpresa agradable — el acceso desde Python *siguió funcionando igual* sobre el controlador original, así que no hubo que sacrificar nada.

Resuelto eso, apareció un segundo problema, más sutil: Windows emparejaba el robot por Bluetooth, pero nunca terminaba de crear el "puerto serie" virtual que el software de LEGO necesita para hablarle (un mecanismo llamado "Standard Serial over Bluetooth link"). Solucionado a mano desde el panel de Bluetooth de Windows. Y encima de eso, un tercer problema: intentar cargar el firmware del robot (el programa base que corre dentro del ladrillo LEGO) se interrumpió a mitad de camino, dejando al robot en un estado de "recuperación" que hace un sonido de tic-tic repetido y no responde — un problema documentado en la comunidad de usuarios de LEGO Mindstorms como el "Síndrome del Ladrillo que Hace Clic". Se resolvió reasignando manualmente el controlador correcto en el Administrador de dispositivos de Windows y reintentando la carga del firmware, que esta vez sí completó con éxito.

Ninguno de estos tres problemas tenía que ver con la teoría cosmo-semiótica ni con un error de diseño del robot: eran, en conjunto, la clase de fricción típica de trabajar con hardware de hace casi veinte años sobre un sistema operativo moderno. Se documentan igual porque consumieron tiempo real y porque las soluciones (sobre todo la del "Síndrome del Ladrillo que Hace Clic") pueden volver a hacer falta en el futuro.

4.2 — Tres bugs reales de sensores, y cómo se encontraron

Con la infraestructura resuelta, se probó cada sensor. Tres de ellos (el multiplexor de sensores traseros, el sensor de proximidad EOPD, y el sensor de Color) daban lecturas sin sentido — atascadas siempre en el mismo valor, sin importar lo que hubiera enfrente. Investigar por qué se convirtió en el trabajo técnico más profundo de la noche.

El multiplexor (SMUX). Este es un componente que permite conectar varios sensores (Touch, Giroscopio, Acelerómetro y Brújula) en un solo puerto del robot. Como la librería de Python usada para hablarle al robot no trae soporte para este multiplexor en particular, se había implementado el protocolo a mano, basándose en el código de otro proyecto de código abierto (ev3dev). El multiplexor no respondía ni siquiera a su registro de identificación más básico. Comparando la dirección de bus usada (un número que identifica a cada dispositivo conectado, como un número de puerta) contra el código fuente real del fabricante del multiplexor, se descubrió que se estaba usando la dirección genérica que usan la mayoría de los sensores HiTechnic, pero el multiplexor específicamente usa una dirección distinta. Corregido ese número, el multiplexor respondió — pero seguía dando datos saturados (valores en el tope de la escala) hasta que se agregó un comando de "detección" que el protocolo necesita antes de empezar a leer canales, y que no se había incluido.

El sensor de proximidad EOPD. Este sensor mide qué tan cerca está un objeto según cuánta luz infrarroja propia rebota de vuelta. Siempre devolvía el mismo valor de "saturado", sin importar qué tan cerca o lejos estuviera un objeto. La causa: el sensor tiene dos modos de sensibilidad

(largo y corto alcance), y ninguno de los dos se estaba activando explícitamente al arrancar — el puerto quedaba en un estado sin configurar, y por eso siempre leía el mismo valor de "tope de escala". Activando el modo correcto (de largo alcance, tras probar el otro y ver que saturaba al revés), el sensor empezó a dar lecturas reales y variables.

El sensor de Color. Este fue, con diferencia, el más difícil de los tres, y el que más tiempo insumió — más de una sesión completa de trabajo, incluyendo un tramo en el que se sospechó (sin razón, como se confirmó después) que el sensor físico estaba dañado. Se probaron, en orden, casi todas las explicaciones razonables: la dirección de bus (correcta), el registro exacto según la documentación disponible (aparentemente correcto), el estado del firmware del robot (se llegó a reinstalar por las dudas), un posible conector flojo (se confirmó que sí influía, pero no explicaba todo), y hasta la posibilidad de que las baterías estuvieran bajas. Nada de eso resolvía el problema del todo. La resolución final llegó revisando código fuente de un proyecto completamente distinto — **leJOS**, un entorno que permite programar el mismo robot en lenguaje Java — que Alexis rescató de una carpeta de archivos antiguos guardada en red. Ese código, escrito y probado por otra persona hace más de quince años para el mismo modelo exacto de sensor, mostraba con total claridad cuál era el error: el valor de color venía en dos partes (dos bytes) que había que combinar en un orden específico ("el byte más importante primero"), y el programa los estaba combinando al revés. Corregido ese detalle — una sola letra de diferencia en la instrucción de lectura —, el sensor empezó a devolver valores de color reales y variables. Es un buen ejemplo de por qué documentar hardware antiguo con código fuente real (no solo con manuales de usuario) tiene valor años después.

4.3 — Las primeras corridas completas: el robot se mueve solo, por primera vez

Con los tres sensores funcionando, se hicieron sucesivas corridas completas del programa — cada vez más largas, terminando la noche con corridas de diez y quince minutos seguidos, sin cable, con todos los sensores, la deliberación, y los motores funcionando a la vez. Cada corrida generó un archivo de registro (datalog) con el detalle de cada ciclo: qué midió cada sensor, qué decidió la mente, qué hizo el motor, y cómo le fue. Estas corridas no fueron solo una demostración: revelaron varios problemas de comportamiento que ningún análisis de escritorio podía haber anticipado, y que se fueron corrigiendo *con los datos en la mano*, siguiendo la filosofía explícita de este proyecto: construir completo primero, corregir después con evidencia real, en vez de intentar probar y perfeccionar cada pieza por separado antes de avanzar.

El robot quedó atascado contra un obstáculo, sin retroceder. En una de las primeras corridas largas, el registro mostró que el robot quedó "clavado" durante 63 ciclos seguidos (más de un minuto y medio) muy cerca de un mueble, disparando una y otra vez la reacción de emergencia sin lograr alejarse de verdad. La causa: el sensor ultrasónico, como casi todos los sensores de este tipo, tiene una "zona muerta" a muy corta distancia — cuando el objeto está demasiado cerca, el eco de sonido no rebota de manera confiable, y el sensor empieza a alternar entre lecturas cortas y falsos "no hay nada cerca". El umbral de distancia que decide cuándo el robot debe frenar y retroceder estaba fijado en un valor (8 centímetros) que caía justo dentro de esa zona muerta, así que la señal de "cuidado" casi nunca llegaba de forma confiable. Se subió ese umbral, primero a 20 y después a 25 centímetros, dejando margen para que la lectura siga siendo confiable antes de que el robot esté realmente pegado a algo.

El retroceso de emergencia era débil, y a veces no alcanzaba para escapar. Incluso con el umbral corregido, en una corrida posterior el robot volvió a quedar atrapado muchos ciclos seguidos, retrocediendo y girando un poco cada vez pero sin separarse lo suficiente del obstáculo. Acá se tomó una decisión de diseño importante, pedida explícitamente por Alexis: en vez de simplemente programar "si llevás muchos vetos seguidos, aplicá más potencia" como una fórmula mecánica externa, se conectó la intensidad de esa respuesta a la *misma señal de malestar interno* que ya calculaba el organelo de propiocepción — específicamente, una señal ya prevista en el diseño original llamada "presión de desacople" (la urgencia de salir de una situación de la que no se puede escapar). Cada vez que el robot queda atrapado, esa presión sube, y la potencia y duración reales del retroceso salen de esa presión, no de un contador aparte. Es la diferencia entre "el robot ejecuta un patrón de escape programado" y "el robot empuja cada vez más fuerte porque, literalmente, cada vez le urge más salir". Probado en corridas posteriores, el mismo tipo de situación se resolvió en pocos ciclos en vez de decenas.

El robot casi no se movía por sí mismo. En una corrida larga, se notó que el rumbo medido

por la brújula quedaba igual durante decenas de ciclos seguidos, aunque el robot estuviera "decidiendo" moverse en cada uno. La sospecha, confirmada después: la potencia mínima permitida para los movimientos normales (40 sobre 100) probablemente quedaba por debajo de la fuerza real necesaria para vencer la fricción de las ruedas contra el piso — sobre todo en los giros suaves, donde una rueda recibe la mitad de esa potencia ya de por sí baja. El motor recibía la orden y "zumbaba", pero el robot apenas se desplazaba. Subiendo el piso de potencia (de 40–70 a 75–90), las corridas siguientes mostraron un cambio de rumbo real en más del 80% de los ciclos, contra menos del 20% antes — el robot pasó de moverse casi en el lugar a explorar de verdad.

Un segundo sensor de cercanía, para cubrir el punto ciego del primero. Como el sensor ultrasónico tiene esa "zona muerta" a corta distancia, se agregó al EOPD (que mide luz reflejada, no eco de sonido, y por lo tanto no comparte esa limitación) como un segundo disparador independiente de la misma reacción de emergencia. Así, si el ultrasónico falla justo cuando más se lo necesita, el EOPD puede cubrir ese hueco.

Un nuevo sentido: la energía propia. A pedido de Alexis, se agregó un organelo nuevo que lee el voltaje real de la batería del robot y lo convierte en una señal de "energía" entre 0 y 1. Esta señal no frena al robot ni lo obliga a nada — se limitó a alimentar exactamente el mismo lugar del diseño original de propiocepción que ya estaba previsto para recibir esta información (y que hasta esta noche había quedado vacío, por honestidad, ya que nadie lo calculaba todavía). La decisión de diseño fue explícita: el robot debe *saber* cuánta energía tiene, como una sensación más de su propio cuerpo, pero qué hacer con ese saber — explorar más o menos — queda para la deliberación, no para un corte de seguridad automático.

El misterio del giro hacia la derecha. Sobre el final de la noche, Alexis notó que el robot parecía preferir sistemáticamente girar hacia la derecha. Revisando el código, se confirmó que no hay ninguna preferencia escrita — las opciones de giro son perfectamente simétricas. Pero analizando los 20 registros de actividad de toda la noche en conjunto, se encontró que, del total de decisiones tomadas, un 58% fueron giros hacia la derecha contra un 26% hacia la izquierda — un sesgo real, no imaginado. La explicación más probable, y que queda pendiente de confirmar en una próxima sesión, combina dos cosas: primero, la mente del robot le da un pequeño "bono" fijo a repetir la última decisión tomada (para evitar que el robot haga zigzag sin sentido), y como la memoria de experiencia aprende muy lentamente, ese bono de repetición domina largos tramos de cada corrida una vez que, por puro azar al arrancar, se elige un lado. Segundo — y esto habría que medirlo con los motores en el banco — es posible que exista una asimetría física real entre los dos motores del robot (uno un poco más fuerte o con más fricción que el otro), que hace que girar hacia un lado le resulte, en la práctica, más efectivo que girar hacia el otro, reforzando aún más ese sesgo con el correr de cada corrida.

4.4 — Dos mejoras de "cuidado del organismo", no de comportamiento

Se agregaron, además, dos detalles que no cambian cómo decide el robot, sino cómo se lo puede seguir en la práctica:

- Un sonido al empezar y otro distinto al terminar cada corrida, para saber sin mirar la pantalla cuándo el robot está realmente activo.
- Un cierre del programa "a prueba de fallos": si algo se corta a mitad de camino (como pasó dos veces esta noche, cuando la batería se agotó de golpe durante una corrida), cada paso de la limpieza final queda protegido por separado, de modo que aunque uno falle, los demás — sobre todo el que guarda el archivo de registro en disco — se ejecuten igual. Antes de este cambio, una falla a mitad de la limpieza final podía hacer perder el registro completo de toda la corrida; ambas veces que se puso a prueba esta noche, el registro se salvó.

5. Resumen de bugs reales corregidos

Componente	Síntoma	Causa raíz y corrección
Multiplexor (SMUX)	No respondía, ni a su identificación	Dirección de bus incorrecta (se usaba la genérica en vez de la propia del multiplexor); además faltaba el comando de detección antes de leer canales.
Sensor EOPD	Siempre "saturado", sin variar con la distancia	Nunca se activaba el modo de sensibilidad del sensor al arrancar; el puerto quedaba sin configurar.
Sensor de Color	Siempre en cero, sin variar con ningún color	Los dos bytes del valor de color se combinaban en el orden equivocado (se corrigió comparando contra código fuente real de otro proyecto, leJOS).
Capa reactiva (umbral)	El robot quedaba pegado a obstáculos sin retroceder	El umbral de distancia de seguridad caía dentro de la "zona muerta" de corto alcance del sensor ultrasónico; se subió el umbral.
Retroceso de emergencia	A veces no alcanzaba para separarse de un obstáculo	Se conectó la fuerza del retroceso a la señal interna de "presión de desacople" (malestar), en vez de dejarla fija.
Movimiento normal	El robot casi no se desplazaba, aunque "decidía" moverse	La potencia mínima permitida no alcanzaba para vencer la fricción real; se subió el piso de potencia.
Registro de datos ante fallas	Se perdía toda la corrida si algo fallaba al final	Se protegió cada paso de la limpieza final por separado.

6. Lo que aprendimos de lo que Sí salió bien

No todo lo relevante de esta sesión fue corregir errores. Varias decisiones de diseño, tomadas en esta sesión o en la anterior, se pusieron a prueba esta noche con datos reales y se confirmaron como acertadas:

- **Barajar el orden de evaluación de las opciones de movimiento** (una corrección de la sesión anterior) siguió demostrando su valor: sin eso, la mente del robot elegiría siempre la misma opción por defecto al arrancar con la memoria vacía, contradiciendo la idea central de la teoría de que el sistema debe introducir diferencia cuando el mundo no cambia.
 - **Construir el programa completo primero, y corregir después con datos reales** — la decisión explícita de Alexis desde el principio de este proyecto — funcionó exactamente como se esperaba: casi todos los bugs reales de esta sesión (el umbral de la capa reactiva, la potencia mínima, la necesidad de un segundo sensor de cercanía) solo se pudieron descubrir *viendo al robot moverse de verdad*, no leyendo el código de antemano.
 - **Separar la capa reactiva de la deliberación** demostró su valor: incluso durante las dos caídas de batería de esta noche, el robot nunca dejó de proteger sus datos ni de intentar frenar ante un obstáculo hasta el último instante posible — la protección física no depende de que la "mente" esté funcionando bien.
 - **Desconfiar de las explicaciones fáciles cuando un sensor falla.** El sensor de Color pasó, en distintos momentos, por sospechas de batería, de firmware, de puerto, y hasta de estar roto — todas parcialmente plausibles, ninguna correcta del todo. La causa real solo apareció cruzando fuentes independientes (documentación oficial de 2006, el driver de otro sistema operativo, y finalmente código fuente de otro lenguaje de programación completamente distinto). La lección para el futuro: ante un sensor que "no tiene sentido", vale la pena buscar una segunda implementación independiente y comparar línea por línea, en vez de seguir ajustando parámetros a ciegas.
 - **Un conector flojo puede imitar perfectamente un error de software.** Más de una vez esta noche, un sensor que fallaba de forma "rara" (a veces sí, a veces no, sin patrón) resultó ser, simplemente, un cable mal asentado. Vale la pena revisar lo físico antes de asumir que el código está mal.
-

7. Estado del robot al cierre de esta sesión

Funcionando y confirmado con datos reales: conexión Bluetooth sin cable estable; sensor ultrasónico; multiplexor completo (Touch, Giroscopio, Acelerómetro, Brújula); sensor de Color HiTechnic; sensor EOPD; capa reactiva con doble disparador de cercanía (ultrasónico + EOPD) y detección de choque trasero; deliberación con memoria y veto por trauma; propiocepción con energía real de batería; corridas limpias de hasta quince minutos seguidos sin intervención.

Pendiente para una próxima sesión: medir si los dos motores entregan realmente la misma fuerza ante la misma orden (para entender mejor el sesgo hacia la derecha); calibrar con más precisión las escalas de cada sensor dentro del cálculo de "cuánto cambió el mundo"; considerar integrar el ciclo del robot con el motor genérico ya vendorizado del proyecto ANIMA, en vez del bucle actual escrito a mano en main.py (una mejora de fidelidad arquitectónica, no urgente).

8. Glosario breve, para quien no programa

- **Organelo:** cada módulo de código que representa una capacidad del robot (un sensor, el motor, la mente, etc.), pensado como un "órgano" con una función propia y clara.
- **Valencia:** memoria de "cómo le fue" al robot con cada opción de movimiento en el pasado — se refuerza con buenos resultados, se castiga con malos.
- **CambioTotal:** una medida de cuánto cambió el estado general del robot (combinando todos los sensores a la vez) de un instante al siguiente.

- **Propiocepción:** la capacidad del robot de "sentir" su propio estado interno (bienestar, malestar, energía), no solo el mundo externo.

- **Veto reactivo:** una frenada y retroceso automáticos, sin pasar por la deliberación, ante un peligro físico inmediato (un obstáculo muy cerca o un golpe).

- **Presión de desacople:** señal interna que sube cuando el robot no logra escapar de una situación, usada esta noche para hacer que la respuesta de emergencia sea cada vez más fuerte.

- **I2C / Bluetooth RFCOMM:** dos protocolos de comunicación de bajo nivel — el primero para hablarle a sensores conectados por cable dentro del robot, el segundo para la conexión inalámbrica entre la computadora y el robot.

- **Datalog:** archivo de registro (en formato CSV, abrible en Excel) donde queda guardado, ciclo a ciclo, todo lo que el robot midió, decidió e hizo durante una corrida.

9. Dónde están guardadas las cosas

- **Registros de actividad de esta sesión (20 archivos):** D:\Cosmolab\Cosmorobot\data\log\, nombrados por fecha y hora (sesion_AAAAMMDD_HHMMSS.csv).
- **Código de cada órgano del robot:** D:\Cosmolab\Cosmorobot\organelos\.
- **Parámetros ajustables (umbrales, potencias, escalas):** D:\Cosmolab\Cosmorobot\config\.
- **Este documento:** D:\Cosmolab\Cosmorobot\docs\.

Documento generado el 11 de julio de 2026, al cierre de la sesión de construcción.